

Design an E-Commerce System using Microservices.

◆ Services you should identify:

- User Service
- Product Service
- Order Service
- Payment Service
- Inventory Service
- Notification Service
- Configuration service
- Gateway service

◆ Architecture (High level):

```
Client → API Gateway → Microservices → Databases  
→ Message Broker (Kafka/RabbitMQ)
```

◆ Key design points:

- Each service has **its own database**
- Sync communication → REST/gRPC
- Async communication → Events
- API Gateway for routing & auth
- Circuit breaker for failures

◆ Follow-ups:

- How do you handle payment failure? → **Saga pattern**

- How do services communicate? → REST + Events
- How do you scale? → Independent scaling via containers

Design Microservices Communication

Q How do microservices communicate with each other?

Microservices communicate with each other using **well-defined APIs and messaging mechanisms**, rather than direct database access or shared memory. The most common approach is **synchronous communication**, where one service calls another using protocols like **HTTP/REST** or **gRPC**, receiving an immediate response; this is simple and widely used for request–response scenarios.

Another important approach is **asynchronous communication**, where services exchange messages through **message brokers** such as Kafka, RabbitMQ, or cloud queues; this allows services to work independently, improves scalability, and reduces tight coupling. To manage service discovery and routing, microservices often use **service registries**, **API gateways**, or **service meshes**, which handle concerns like load balancing, retries, security, and observability.

Overall, microservice communication is designed to be **loosely coupled, resilient, and independent**, so that each service can evolve, scale, and fail without impacting the entire system.

Q When to use which kind of communication?

Use **synchronous communication** when you need an instant reply to continue work. It is like a phone call—you ask a question and wait for the answer. This is good for login, fetching data, or validations, where the user is waiting for a result.

Use **asynchronous communication** when you don't need an immediate reply. It is like sending a message or email—the work happens later. This is best for background tasks, notifications, reports, and events, where speed and reliability matter more than instant response.

Q What is REST?

REST (Representational State Transfer) is a communication style where microservices talk to each other using HTTP methods like GET, POST, PUT, DELETE. Data is usually sent in JSON format, making it easy to read and use. REST is simple, flexible, and widely used, especially for web and mobile applications.

Q When would you use REST in microservices?

Use REST when simplicity, readability, and broad client support are important, such as public APIs or user-facing services.

Q What is gRPC?

gRPC is a high-performance communication framework that uses HTTP/2 and Protocol Buffers (binary format) instead of JSON. It is faster and more efficient than REST and supports bi-directional streaming, making it ideal for internal microservice communication.

Q: When would you prefer gRPC over REST?

A: Use gRPC when you need high speed, low latency, and strong contract-based communication between internal services.

REST vs gRPC (One-Line Summary)

- **REST:** Easy, human-readable, best for external APIs
- **gRPC:** Fast, efficient, best for internal service-to-service communication.

Design a Fault-Tolerant Microservices System

Q How do you design microservices to handle failures?

Microservices are designed to handle failures by expecting things to go wrong and reducing their impact. Each service is kept independent, so if one fails, others keep working. **Timeouts**, **retries**, and **circuit breakers** are used to prevent one failing service from affecting the whole system.

Fallbacks and cached data help return a response even when a service is down, and asynchronous communication allows work to continue without waiting. Proper logging and monitoring ensure failures are detected and fixed quickly, so the system fails gracefully instead of breaking completely.

Circuit Breaker

A circuit breaker stops calling a service that is already failing. If a service keeps giving errors, the circuit opens and further requests are blocked for some time. This prevents system overload and gives the service time to recover.

Retry with Backoff

Retry with backoff means trying the request again after a failure, but with a small delay that increases each time. This avoids hitting the failing service again and again at the same time, reducing pressure on the system.

Timeout

A timeout defines how long a service should wait for a response from another service. If the response does not come in that time, the request fails fast. This prevents threads and resources from getting stuck.

Fallback

A fallback is a backup response used when a service fails. Instead of returning an error, the system may return cached data, a default value, or a friendly message, keeping the user experience smooth.

Bulkhead Pattern

The bulkhead pattern isolates different parts of the system into separate resource pools. If one service or feature fails and consumes all resources, it does not affect other services—similar to compartments in a ship preventing it from sinking.

Example:

If the Payment Service fails, the circuit breaker opens, so the Order Service stops sending requests to it. The Order Service uses a fallback, such as marking the order as “*Payment Pending*” instead of failing the entire flow. Once the Payment Service recovers, it processes the payment later, and the system becomes eventually consistent, meaning all services reach the correct final state after some time, not instantly.

Design Distributed Transaction Handling

Q How do you manage transactions across microservices?

Transactions in microservices are managed using the **Saga pattern**, where a transaction is split into multiple **series of local transactions**, across services. Each service updates its own database and publishes events. If something fails, **compensating actions** are executed. This approach follows **eventual consistency** instead of strict ACID transactions.

Q Compensating Actions

Compensating actions are **undo operations** used when a step in a microservice transaction fails. Since microservices do not use a single database transaction, each service performs its own local transaction. If a later step fails, compensating actions reverse the earlier successful steps, for example **canceling an order, refunding a payment, or releasing reserved inventory**.

Q Types of Saga

Choreography

In choreography, there is **no central controller**. Each service listens to events and reacts by performing its action and publishing the next event. This approach is **simple and loosely coupled**, but the overall flow can be hard to understand as the system grows.

Orchestration

In orchestration, a **central Saga Orchestrator** controls the transaction flow. It tells each service what action to perform and handles failures by triggering compensating actions. This makes the flow **easy to manage and monitor**, but adds some coupling through the orchestrator.

Interview Tip (One Line):

Saga ensures consistency in microservices using local transactions and compensating actions instead of distributed transactions.

Design Database Strategy for Microservices

In microservices, the database strategy follows “**database per service**”, where each microservice owns and manages its **own database**. This ensures **loose coupling, independent scaling, and better fault isolation**. Services communicate through **APIs or events**, not by directly accessing each other’s databases. Data consistency is handled using **eventual consistency and Saga patterns** instead of shared transactions. This approach allows each service to choose the **best database type** (SQL or NoSQL) based on its needs.

🔍 **Loose Coupling:** Each microservice works independently and communicates only through APIs or events, so changes in one service do not break others.

🔍 **Independent Scaling:** Each service can be scaled separately based on its load, without scaling the entire system.

🔍 **Schema Evolution:** Each service can change or update its database schema without impacting other services, enabling faster development.

🔍 **Fault Isolation:** If one service or its database fails, it does not bring down the whole system; other services continue to function.

In microservices, **joins are not done at the database level** like in monolithic systems. Instead:

👉 **API Calls:** One service fetches required data from another service using APIs and combines the response at the application level. This is useful for **real-time reads** but should be used carefully to avoid latency and tight coupling.

👉 **Event Materialization:** Services listen to events from other services and store a **local copy of required data** in their own database. This allows fast reads without cross-service calls and follows **eventual consistency**.

Interview one-liner:

Joins in microservices are handled at the **service level using APIs** or by **pre-building data using events**, not by shared databases.

Design API Gateway

Why do we need an API Gateway?

An **API Gateway** is needed to act as a **single entry point** for all client requests in a microservices system. It routes requests to the correct microservice and hides internal service details from clients. The API Gateway also handles **cross-cutting concerns** such as **authentication, authorization, rate limiting, request validation, logging, and caching**, so individual services can stay simple and focused on business logic. It can also **aggregate responses** from multiple services, reducing the number of client calls and improving performance and security.

Responsibilities:

- ❓ **Authentication:** Verifies **who the user is** before allowing access.
- ❓ **Authorization:** Checks **what the user is allowed to do** after authentication.
- ❓ **Routing:** Forwards client requests to the **correct microservice**.
- ❓ **Rate Limiting:** Restricts the number of requests to **prevent overload or abuse**.

🔍 **Aggregation:** Combines responses from **multiple services** into **one** response for the client.

Tools:

- **Spring Cloud Gateway:** A **Java-based API Gateway** used with Spring Boot microservices. It provides routing, authentication, rate limiting, and filters, and is easy to integrate in Spring ecosystems.
- **NGINX:** A **high-performance reverse proxy** and load balancer. It is fast, lightweight, and commonly used as an API Gateway for routing, SSL termination, and rate limiting.
- **Kong:** An **open-source, cloud-native API Gateway** built on NGINX. It offers powerful plugins for authentication, security, analytics, and monitoring, and works well in Kubernetes environments.

Design Service Discovery

Services find each other using service discovery instead of hard-coded IPs or URLs. Each microservice registers itself with a service registry (like Eureka or Consul) when it starts. When one service needs to call another, it queries the registry to get the available service instances and then routes the request using load balancing. This allows services to scale up or down dynamically and communicate reliably without knowing each other's exact network details.

🔍 **Client-side discovery (Eureka):**

Each service **registers itself** with a service registry like **Eureka**. The calling service **asks Eureka** for the target service's address and then directly calls one of its instances. The client is responsible for discovery and load balancing.

❓ **Server-side discovery (Kubernetes):**

Services are discovered by the **platform itself**. The client sends a request to a **stable service name**, and Kubernetes (via Service and DNS) **routes and load-balances** the request to the correct pod. The client does not need to know where the service is running.

◆ **Interview Tip:**

“In Kubernetes, service discovery is built-in.”

Design Security in Microservices

Microservices are secured by applying security at multiple layers. First, authentication and authorization are centralized using an API Gateway with standards like JWT or OAuth2, ensuring only valid users and services can access APIs. Service-to-service security is handled using mutual TLS (mTLS) or secure tokens to encrypt communication and verify identities. Sensitive data is protected using encryption (in transit and at rest), and access is controlled using least-privilege roles. Finally, rate limiting, monitoring, and logging help detect and prevent attacks, making the system secure and reliable.

- **API Gateway Authentication:** All external requests pass through the API Gateway, which handles authentication and blocks unauthorized access before requests reach microservices.
- **JWT / OAuth2:** JWT tokens and OAuth2 are used to **verify user identity and permissions** in a stateless and secure way across services.

- **mTLS for Internal Calls:** Mutual TLS encrypts **service-to-service communication** and ensures that only trusted services can talk to each other.
- **Secrets Management:** Sensitive data like passwords, API keys, and certificates are stored securely using tools like **Vault or Kubernetes Secrets**, not hard-coded in code.

Interview one-liner:

Microservices are secured using gateway-based auth, token standards, encrypted internal communication, and secure secrets storage.

Design Logging & Monitoring

How do you monitor microservices?

Microservices are monitored using centralized observability tools. Each service exposes metrics (like CPU, memory, request count, and latency), which are collected by monitoring systems such as Prometheus. Centralized logging is used to gather logs from all services in one place for easy debugging. Distributed tracing helps track a request as it flows across multiple services to find performance issues or failures. Dashboards and alerts are set up to quickly detect problems and keep the system healthy.

🔍 **Centralized Logging (ELK):** All microservice logs are collected in one place using the ELK stack (Elasticsearch, Logstash, Kibana), making it easy to search and debug issues.

🔍 **Metrics (Prometheus):** Prometheus collects system and application metrics like CPU usage, memory, request count, and latency to monitor performance and health.

🔍 **Tracing (Jaeger):** Jaeger is used for distributed tracing, allowing you to follow a request across multiple services and identify bottlenecks or failures.

Design Microservices Deployment

How do you deploy microservices?

Microservices are deployed using containerization and orchestration. Each service is packaged as a Docker container, ensuring consistency across environments. Containers are then deployed using Kubernetes, which handles service discovery, scaling, load balancing, and self-healing. CI/CD pipelines automate building, testing, and deploying services, enabling frequent and reliable releases. This approach allows each microservice to be deployed independently without affecting others.

- **Docker Containers:**

Each microservice is packaged into a Docker container that includes the application and its dependencies. This ensures the service runs **the same way in development, testing, and production**.

- **Kubernetes Orchestration:**

Kubernetes runs and manages these containers. It handles **deployment, scaling, load balancing, service discovery, and self-healing** (restarting failed containers) automatically.

- **CI/CD Pipelines:**

CI/CD pipelines automate the process of **building, testing, and deploying** microservices. When code is pushed, the pipeline builds a Docker image, runs tests, and deploys the service to Kubernetes, enabling **fast and safe releases**.

Design a Scalable Notification System

Components

- **Notification Service:** Receives notification requests and decides the channel (Email/SMS).
- **Message Queue:** Buffers notification events and processes them asynchronously.
- **Email/SMS Providers:** External services that actually send messages to users.

Design (Relating to Patterns)

The system works using **asynchronous events**, where the Notification Service publishes a message to the queue instead of sending emails or SMS directly. This follows the **event-driven pattern**, making the system scalable and non-blocking.

A **retry mechanism** is used to handle temporary failures from email or SMS providers; if sending fails, the message is retried with backoff. If a message fails even after multiple retries, it is moved to a **Dead Letter Queue (DLQ)**.

The DLQ stores failed messages for later inspection or reprocessing, ensuring failures do not block the system and improving reliability.

Design Rate Limiting (Interview Friendly)

Question:

How do you protect services from overload?

Answer:

Services are protected from overload by applying **rate limiting at the API Gateway**, which controls how many requests a client can make within a given time. This prevents abuse and ensures fair usage.

Common algorithms include the **Token Bucket**, where requests are allowed only if tokens are available (allowing short bursts), and the **Leaky Bucket**, where requests are processed at a fixed rate, smoothing traffic. Rate limiting helps maintain system stability and consistent performance under high load.